

Odoo 17 Production HA Cluster — Final Runbook

olilaerp.com

Primary: `103.245.204.6` Standby: `43.231.78.35` Nginx + Monitoring + repmgr

Witness: `81.0.220.76` Domain: `olilaerp.com` Database: `olila_glassware` Filestore
(identical path both servers): `/home/olila/.local/share/Odoo/filestore/`

⚠ READ THIS FIRST — Pre-Flight Safety Checklist

Do not skip any item below. Each one caused a real problem in earlier testing.

0.1 Take a full backup before touching anything

```
bash

# on Primary (103.245.204.6)
sudo -u postgres pg_dump -Fc olila_glassware > /root/olila_glassware_$(date +%F)
sudo tar -czf /root/filestore_backup_$(date +%F).tar.gz /home/olila/.local/shar
```

Copy both files somewhere *off* both servers (your local machine, S3, another VM) before proceeding.

0.2 Audit for existing sync jobs on the filestore path

You mentioned rsync may already be running against this same path on both servers. Two syncs writing to/from the same directory simultaneously can cause loops or data loss.

Check **both** servers:

```
bash

crontab -l
sudo crontab -l -u odoo 2>/dev/null
ps aux | grep -i rsync
ps aux | grep -i lsyncd
sudo systemctl status lsyncd 2>/dev/null
```

If you find an existing rsync cron job or lsyncd instance already syncing this path:

- **Disable it before setting up the new one** (comment out the cron line, or `sudo systemctl stop lsyncd && sudo systemctl disable lsyncd`)).
- Note which **direction** it was syncing (which server was source vs target) — if it was already Primary → Standby, that's consistent with the plan below and just needs replacing with lsyncd for real-time behavior. If it was bidirectional or Standby → Primary, stop it immediately — that's actively dangerous once real replication is in place, since it could push stale/standby data back onto the primary.

0.3 Confirm identical PostgreSQL major version on both servers

```
bash
```

```
# on BOTH servers
pg_lsclusters
```

Both must show the same major version number (e.g. both `17`). If they differ, install the matching version before proceeding — streaming replication will not work across major versions.

0.4 Confirm identical OS user and Postgres role naming

This was the single biggest source of errors last time — mismatched service/role names (`odoo` vs `odoo17`) caused hours of auth failures.

```
bash

# on BOTH servers
sudo systemctl list-units --type=service | grep -i odoo
getent passwd odoo    # or whatever the actual service user is
```

Decide now, before continuing: pick one OS username and one Postgres role name, and use the exact same one on both servers. This guide assumes both are `odoo` — adjust every command below if yours differs, but keep it **identical on both sides**.

0.5 Confirm the actual filestore path is correct on both servers

```
bash

# on BOTH servers
sudo grep data_dir /etc/odoo*.conf
ls -la /home/olila/.local/share/0doo/filestore/
```

Confirm this path actually contains the real filestore (should have a subfolder named after `olila_glassware`) on the Primary, and check what currently exists there on Standby before you overwrite anything.

Stop here and confirm all five checks above before continuing.

PART 1 — SSH Trust Between All Three Servers

On the **Primary**, as the `odoo` OS user:

```
bash

getent passwd odoo    # confirm home directory path, use it below — assumed /home
sudo mkdir -p /home/olila/.ssh
sudo chown odoo:odoo /home/olila/.ssh
sudo chmod 700 /home/olila/.ssh

sudo -u odoo ssh-keygen -t ed25519 -N "" -f /home/olila/.ssh/id_ed25519
```

Copy the **public key** manually (safer than `ssh-copy-id`), avoids the scratch-directory issue

from before):

```
bash
sudo -u odoo cat /home/olila/.ssh/id_ed25519.pub
```

On the **Standby**, append that exact output:

```
bash
sudo mkdir -p /home/olila/.ssh
sudo chown odoo:odoo /home/olila/.ssh
sudo chmod 700 /home/olila/.ssh
sudo -u odoo bash -c 'echo "PASTE_PUBLIC_KEY_HERE" >> /home/olila/.ssh/authorized_keys'
sudo chmod 600 /home/olila/.ssh/authorized_keys
sudo chown odoo:odoo /home/olila/.ssh/authorized_keys
```

Test before moving on:

```
bash
# on Primary
sudo -u odoo ssh -i /home/olila/.ssh/id_ed25519 odoo@43.231.78.35 "echo connect"
```

Must return `connection ok` with no password prompt.

PART 2 — PostgreSQL Streaming Replication

2.1 On Primary — enable replication

```
bash
sudo -u postgres psql -c "SHOW config_file;" # find the real path, use it below
```

Edit `postgresql.conf`:

```
conf
listen_addresses = '*'
wal_level = replica
max_wal_senders = 5
wal_keep_size = 1024
hot_standby = on
wal_log_hints = on
archive_mode = on
archive_command = 'cp %p /var/lib/postgresql/wal_archive/%f'
```

`wal_log_hints = on` is set now, deliberately — this is required later for `pg_rewind` and cannot be added after the fact without a restart.

```
bash
```

```
sudo mkdir -p /var/lib/postgresql/wal_archive  
sudo chown postgres:postgres /var/lib/postgresql/wal_archive
```

Edit `pg_hba.conf` — **check what's already in this file first**, don't blindly append (dead/duplicate rules caused real confusion last time):

```
bash
```

```
sudo cat -n /etc/postgresql/*/main/pg_hba.conf
```

Add these lines cleanly (avoid duplicating anything already present):

```
conf
```

host	replication	replicator	43.231.78.35/32	md5
host	replication	replicator	81.0.220.76/32	md5

Create the replication role:

```
bash
```

```
sudo -u postgres psql -c "CREATE ROLE replicator WITH REPLICATION LOGIN ENCRYPT
```

Generate a real strong password rather than guessing one:

```
bash
```

```
openssl rand -base64 24
```

```
bash
```

```
sudo systemctl restart postgresql
```

2.2 On Standby — clone the primary

```
bash
```

```
sudo systemctl stop postgresql  
sudo systemctl stop odoo # make sure Odoo can't touch this DB during the clone  
sudo mv /var/lib/postgresql/17/main /var/lib/postgresql/17/main.old
```

```
bash
```

```
sudo -u postgres pg_basebackup -h 103.245.204.6 -D /var/lib/postgresql/17/main  
-U replicator -P -v -R -X stream -C -S standby1
```

Run this in the foreground and let it finish completely — do not background it or leave the

terminal session in a way that could suspend it (a suspended `pg_basebackup` process caused real problems last time). If your connection might drop, use `screen` or `tmux`:

```
bash

sudo apt install tmux -y
tmux new -s basebackup
# then run the pg_basebackup command inside the tmux session
```

Confirm `postgresql.auto.conf` was written correctly:

```
bash

sudo cat /var/lib/postgresql/17/main/postgresql.auto.conf
```

Should show `primary_conninfo` pointing at `103.245.204.6` with the `replicator` user.

Add `application_name` **explicitly** — this was the actual root cause of a repmgr registration failure last time:

```
bash

sudo -u postgres psql -c "ALTER SYSTEM SET primary_conninfo = 'host=103.245.204
```

Also set `wal_log_hints` here too:

```
conf

wal_log_hints = on
```

```
bash

sudo systemctl start postgresql
```

2.3 Verify replication is actually working

```
bash

# on Primary
sudo -u postgres psql -c "SELECT application_name, client_addr, state, sync_sta
```

Expect exactly **one row**: `application_name = standby_node`, `client_addr = 43.231.78.35`, `state = streaming`.

```
bash

# on Standby
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"

Must return (t).
```

PART 3 — Filestore Real-Time Sync (lsyncd)

3.1 Install lsyncd on the Primary only (source of sync)

```
bash

sudo apt install lsyncd -y
```

3.2 Config

/etc/lsyncd/lsyncd.conf.lua:

```
lua

settings {
    logfile      = "/var/log/lsyncd/lsyncd.log",
    statusFile   = "/var/log/lsyncd/lsyncd-status.log",
    statusInterval = 5,
}

sync {
    default.rsync,
    source       = "/home/olila/.local/share/Odoo/filestore/",
    target       = "odoo@43.231.78.35:/home/olila/.local/share/Odoo/filestore/",
    delete      = true,
    rsync        = {
        archive = true,
        compress = true,
        whole_file = false,
        rsh = "/usr/bin/ssh -i /home/olila/.ssh/id_ed25519 -o StrictHostKeyChecki
        _extra = {"--partial", "--inplace"}
    }
}
```

Confirm the target directory exists and is owned correctly on Standby first:

```
bash

# on Standby
sudo mkdir -p /home/olila/.local/share/Odoo/filestore
sudo chown -R odoo:odoo /home/olila/.local/share/Odoo/filestore
```

Test the SSH connection manually before starting lsyncd:

```
bash

sudo -u odoo ssh -i /home/olila/.ssh/id_ed25519 odoo@43.231.78.35 "echo ok"
```

Then:

```
bash
```

```
sudo systemctl enable --now lsyncd
sudo journalctl -u lsyncd -f
```

Watch the first sync happen — confirm files actually transfer, then Ctrl+C.

Do not run lsyncd (or any sync) in the reverse direction while Primary is active — that's what caused the "same path on both, may already be syncing" ambiguity you flagged. Only one direction should ever be live at a time (see Part 5 for the fail-back exception).

PART 4 — Odoo Service State

Primary: `odoo` service enabled and running normally.

Standby: `odoo` service must stay **stopped and disabled** until a real failover happens:

```
bash

sudo systemctl stop odoo
sudo systemctl disable odoo
```

Confirm both servers' `/etc/odoo.conf` reference:

- The **same** `db_name = olila_glassware`
- The **same** `db_user` (must be a role that actually exists — verify with `\du` on either server, they're identical since replicated)
- The **same** `db_password`
- `db_host = localhost` (socket) on both

```
bash

sudo -u postgres psql -c "\du"
```

Confirm the role you're using in `odoo.conf` is genuinely listed here before moving on.

PART 5 — nginx (on the third server, `81.0.220.76`)

```
nginx
```

```

upstream odoo_cluster {
    server 103.245.204.6:8069 max_fails=2 fail_timeout=10s;    # PRIMARY — must
    server 43.231.78.35:8069 backup;                        # STANDBY — bac
}

upstream odoo_chat {
    server 103.245.204.6:8072 max_fails=2 fail_timeout=10s;
    server 43.231.78.35:8072 backup;
}

server {
    listen 80;
    server_name olilaerp.com;

    location / {
        proxy_pass http://odoo_cluster;
        proxy_set_header X-Forwarded-Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_redirect off;
    }

    location /websocket {
        proxy_pass http://odoo_chat;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}

```

Test and reload:

```

bash

sudo nginx -t
sudo systemctl reload nginx

```

Once working, add HTTPS via certbot for `olilaerp.com` — ask if you want that added as a separate step; not included here to keep this focused.

PART 6 — repmgr (Automatic Failover)

6.1 Install on all three servers (including the witness)

```

bash

sudo apt install postgresql-17-repmgr -y

```

The witness server (`81.0.220.76`) needs `postgresql-17` installed too (even though it

won't hold real data) since repmgr's witness role runs a minimal Postgres instance for voting purposes.

6.2 Create the repmgr role + database on Primary

```
bash
```

```
sudo -u postgres psql -c "CREATE USER repmgr SUPERUSER LOGIN ENCRYPTED PASSWORD  
sudo -u postgres psql -c "CREATE DATABASE repmgr OWNER repmgr;"
```

Add to `pg_hba.conf` on **all three servers**:

```
conf
```

local	replication	repmgr		trust
host	replication	repmgr	103.245.204.6/32	md5
host	replication	repmgr	43.231.78.35/32	md5
host	replication	repmgr	81.0.220.76/32	md5
local	repmgr	repmgr		trust
host	repmgr	repmgr	103.245.204.6/32	md5
host	repmgr	repmgr	43.231.78.35/32	md5
host	repmgr	repmgr	81.0.220.76/32	md5

```
bash
```

```
sudo systemctl reload postgresql
```

6.3 `.pgpass` on all three servers

Write it in one shot to avoid the heredoc/append issues from before — always overwrite (`>`), never append blindly:

```
bash
```

```
sudo -u postgres bash -c 'cat > ~/.pgpass << "PGPASS"  
103.245.204.6:5432:repmgr:repmgr:REPLACE_WITH_STRONG_PASSWORD  
43.231.78.35:5432:repmgr:repmgr:REPLACE_WITH_STRONG_PASSWORD  
81.0.220.76:5432:repmgr:repmgr:REPLACE_WITH_STRONG_PASSWORD  
PGPASS'  
sudo -u postgres chmod 600 ~/.pgpass  
sudo -u postgres cat ~/.pgpass # verify content immediately, every time
```

Verify non-interactively (must NOT prompt for password) on each server, testing connection to both others:

```
bash
```

```
sudo -u postgres psql "host=103.245.204.6 dbname=repmgr user=repmgr" -c "SELECT  
sudo -u postgres psql "host=43.231.78.35 dbname=repmgr user=repmgr" -c "SELECT  
sudo -u postgres psql "host=81.0.220.76 dbname=repmgr user=repmgr" -c "SELECT 1
```

6.4 Ensure `shared_preload_libraries` is set — on Primary AND Standby (not witness)

```
bash

sudo -u postgres psql -c "SHOW shared_preload_libraries;"
```

Must show `repmgr`. If blank:

```
bash

sudo -u postgres psql -c "ALTER SYSTEM SET shared_preload_libraries = 'repmgr';"
sudo systemctl restart postgresql
sudo -u postgres psql -c "SHOW shared_preload_libraries;" # confirm it actual
```

⚠ Restarting Postgres on the **Primary** briefly interrupts live traffic — schedule this deliberately.

6.5 Configure `repmgr.conf` — Primary (node 1)

`/etc/repmgr.conf`:

```
conf

node_id=1
node_name='primary_node'
conninfo='host=103.245.204.6 user=repmgr dbname=repmgr connect_timeout=2'
data_directory='/var/lib/postgresql/17/main'
promote_command='repmgr standby promote -f /etc/repmgr.conf --log-to-file'
follow_command='repmgr standby follow -f /etc/repmgr.conf --log-to-file --upstr
failover=automatic
priority=100
log_file='/var/log/postgresql/repmgrd.log'
log_level='INFO'
```

```
bash

sudo -u postgres repmgr -f /etc/repmgr.conf primary register
```

6.6 Configure `repmgr.conf` — Standby (node 2)

```
conf
```

```
node_id=2
node_name='standby_node'
conninfo='host=43.231.78.35 user=repmgr dbname=repmgr connect_timeout=2'
data_directory='/var/lib/postgresql/17/main'
promote_command='repmgr standby promote -f /etc/repmgr.conf --log-to-file'
follow_command='repmgr standby follow -f /etc/repmgr.conf --log-to-file --upstr
failover=automatic
priority=90
log_file='/var/log/postgresql/repmgrd.log'
log_level='INFO'
event_notification_command='/etc/repmgr/event_handler.sh %n %e %s "%t" "%d"'
```

```
bash
```

```
sudo -u postgres repmgr -f /etc/repmgr.conf standby register
```

6.7 Configure the witness (node 3, on the nginx/monitoring server)

```
conf
node_id=3
node_name='witness_node'
conninfo='host=81.0.220.76 user=repmgr dbname=repmgr connect_timeout=2'
data_directory='/var/lib/postgresql/17/main'
log_file='/var/log/postgresql/repmgrd.log'
log_level='INFO'
```

```
bash
```

```
sudo -u postgres repmgr -f /etc/repmgr.conf witness register -h 103.245.204.6
```

This is what closes the 2-node split-brain gap — with three voting members, an automatic failover decision requires a majority, so a single lost network link between just Primary and Standby is far less likely to trigger a false promotion.

6.8 Odoo auto-start/stop event hook (Standby only)

```
bash
```

```

sudo mkdir -p /etc/repmgr
sudo tee /etc/repmgr/event_handler.sh << 'EOF'
#!/bin/bash
NODE_ID=$1
EVENT=$2
SUCCESS=$3

if [ "$EVENT" = "standby_promote" ] && [ "$SUCCESS" = "1" ]; then
    systemctl start odoo
    echo "$(date): Node $NODE_ID promoted, Odoo started" >> /var/log/repmgr-event
fi

if [ "$EVENT" = "standby_follow" ] && [ "$SUCCESS" = "1" ]; then
    systemctl stop odoo
    echo "$(date): Node $NODE_ID became standby again, Odoo stopped" >> /var/log/repmgr-event
fi
EOF
sudo chmod +x /etc/repmgr/event_handler.sh

```

6.9 Start repmgrd on Primary and Standby

```

bash

sudo systemctl enable repmgrd
sudo systemctl start repmgrd
sleep 2
ps aux | grep repmgrd    # confirm a REAL process, not just grep matching itself

```

If nothing shows up, check for a stale pidfile before troubleshooting further (this bit us repeatedly):

```

bash

find / -name "repmgrd.pid" 2>/dev/null
sudo rm -f /tmp/repmgrd.pid
sudo systemctl restart repmgrd

```

6.10 Confirm the full cluster

```

bash

sudo -u postgres repmgr -f /etc/repmgr.conf cluster show
sudo -u postgres repmgr -f /etc/repmgr.conf service status

```

Expect 3 nodes: primary (running), standby (running), witness (running).

PART 7 — Testing (do this before trusting it)

1. **Canary record:** create a test contact + attachment on Primary, note its name.

2. Simulate failure:

```
bash

# on Primary
sudo systemctl stop odoo
sudo systemctl stop postgresql
```

3. Watch the standby's log:

```
bash

tail -f /var/log/postgresql/repmgrd.log
```

Expect automatic promotion within seconds to a couple minutes depending on your `reconnect_attempts`/`interval` settings. 4. **Verify:** `curl -I http://olilaerp.com`, then log in and confirm the canary record + attachment are present, and that you can create new records (proving it's genuinely writable now). 5. **Bring Primary back** using `pg_rewind` via `repmgr's node rejoin` — do this in a maintenance window, not live:

```
bash

# on the old Primary, once it's back online
sudo -u postgres repmgr -f /etc/repmgr.conf node rejoin -d 'host=43.231.78.3'
```

Ongoing — Do Not Skip

- Set up alerting on `repmgrd` promotion events (email/webhook from the event handler script) so a failover doesn't happen silently.
- Rotate all the placeholder passwords in this doc (`REPLACE_WITH_STRONG_PASSWORD`) to real, unique, strong values — never leave any set to something guessable.
- Keep independent backups regardless of replication — replication protects against hardware failure, not against accidental deletes or corruption, which replicate instantly to every node including the witness's metadata.
- Test the full failover **and** fail-back cycle at least twice in a low-traffic window before considering this production-ready.